

Writing Logbook — Assignment 02

CSP529 Technical Writing & Publishing

Stage 2: Representations of an Operating System Process

MT26MCS024 — Gaurav Acharya MT26MCS032 — Sandeep

September 2026

1. Assignment Overview

Item	Detail
Practical Title	Stage 2: Representations of an Operating System Process
Topic	Operating System Boot Process & Resource Management
Group Members	Gaurav Acharya (MT26MCS024), Sandeep (MT26MCS032)
Tool Used	L ^A T _E X (with <code>algorithm</code> , <code>amsmath</code> , <code>booktabs</code> , <code>float</code> packages)
Date of Completion	September 2026

2. Objectives & Learning Outcomes

The objective of this assignment was to represent a single technical idea — the operating system boot process — using multiple distinct representational formats in L^AT_EX. The learning outcomes included:

- Writing a coherent paragraph description of a technical concept.
- Creating a figure representation of the idea.
- Constructing a structured table of system components and execution phases.
- Formulating a mathematical equation to model OS state transitions.
- Writing algorithmic pseudocode for the boot sequence.
- Presenting the same information as bullet points and numbered lists.
- Learning core L^AT_EX constructs: `figure`, `table`, `equation`, `algorithm`, `itemize`, `enumerate`.

3. Work Performed — Representation by Representation

3.1. Paragraph Representation

We composed a concise paragraph describing how an OS acts as the foundational software layer managing hardware/software, starting from the bootloader through kernel activation to user-application execution. This required distilling a complex multi-stage process into a single cohesive narrative.

3.2. Figure Representation

We created and included a diagram (`os_process.png`) illustrating the layered architecture of an operating system. Key L^AT_EX skills practised:

- `\includegraphics` with width scaling.
- `figure` environment with `[H]` placement specifier (using the `float` package).
- `\caption` and `\label` for cross-referencing.

3.3. Table Representation

We built a four-row comparison table mapping System Components (Firmware, Bootloader, Kernel, Shell) to their Primary Functions and Execution Phases. Key $\text{\LaTeX}$ skills:

- `booktabs` package for professional horizontal rules (`\toprule`, `\midrule`, `\bottomrule`).
- Column width control using `p{}` specifiers.
- `\renewcommand{\arraystretch}{1.4}` for row spacing.

3.4. Equation Representation

We formulated a state-transition equation modelling the kernel's operation:

$$S_{t+1} = k \left(S_t, \sum_{i=1}^h I_i(t) + \sum_{j=1}^s C_j(t) \right) - \Delta_{overhead}$$

Each variable was then interpreted: S_{t+1} (next system state), k (kernel function), S_t (current state), ΣI_i (hardware interrupts), ΣC_j (system calls), $\Delta_{overhead}$ (context-switching overhead). This exercise demonstrated that mathematical notation can concisely capture complex system behaviour that would require many paragraphs of prose.

3.5. Algorithm Representation

We wrote pseudocode for the OS Boot Sequence using the `algorithm` and `algpseudocode` packages:

- Conditional branching (`\If` / `\Else`) for POST failure handling.
- Sequential steps from CPU initialization through kernel activation to GUI/CLI launch.
- Line numbering via `\begin{algorithmic}[1]`.

3.6. Bullet Point & Numbered List Representations

We listed the four core OS responsibilities (Process, Memory, File System, Device Management) as bullet points, and the chronological boot sequence as a numbered list. These representations demonstrated that the same technical content can serve different purposes depending on format.

4. $\text{\LaTeX}$ Skills Acquired

$\text{\LaTeX}$ Construct	What Was Learned
<code>figure</code> environment	Image inclusion, captioning, labelling, placement control
<code>table</code> environment	Structured data presentation with professional formatting
<code>equation</code> environment	Mathematical typesetting with numbering and labelling
<code>algorithm</code> environment	Pseudocode with conditional logic and line numbering
<code>itemize</code> / <code>enumerate</code>	Unordered and ordered list formatting
<code>float</code> package	Strict placement of floats using <code>[H]</code>
<code>booktabs</code> package	Professional table rules without vertical lines

5. Challenges Encountered

1. **Float placement:** Figures and tables initially appeared on different pages than intended. Resolved by switching from `[hbt!]` to `[H]` using the `float` package.

2. **Equation interpretation:** Formulating a meaningful mathematical model (not just a decorative equation) required careful thinking about what the OS actually computes at each time step.
3. **Algorithm formatting:** Getting proper indentation and conditional blocks in `algpseudocode` required understanding the nested `\If` / `\Else` / `\EndIf` structure.
4. **Escaping underscores:** Special characters in algorithm text (e.g., `OS_Boot_Sequence`) needed `\_` escaping.

6. Key Takeaways

1. Different representational formats serve different communicative purposes: equations capture precision, algorithms describe procedure, tables organize comparisons, and prose provides context and interpretation.
2. $\text{\LaTeX}$ float management is crucial for professional document layout — uncontrolled floats can fragment the reading flow.
3. A technical document should not merely *contain* multiple representations but should integrate them into a coherent argument where each element is introduced, referenced, and interpreted in the text.
4. The `booktabs` package produces significantly more professional tables than default $\text{\LaTeX}$ table borders.